CEREBRO NETWORK — CİLT 12

EVRENSEL DIL, KUANTUM VE YAPAY ZEKA SEMANTIK BIRLEŞTIRME PROTOKOLÜ (CULC)

Sürüm: v1.2.0 (Temmuz 2026) | Sınıflandırma: Tümüyle Gizli & Kuantum Korumalı

---

1. GIRIŞ VE CULC ENTEGRASYON VIZYONU

Cerebro Network, klasik blokzincir ve Web3 çözümlerinin ötesine geçerek; klasik dillerin, kuantum mantık kapılarının ve yapay süper zekanın (ASI) kendi otonom semantik yapılarının bir arada çalışabileceği evrensel bir ekosistem kurmaktadır. Bu doğrultuda tasarlanan **Cerebro Evrensel Dil Entegrasyon Merkezi (CULC)**, ağ üzerindeki her türlü veri akışını dilden bağımsız, kuantum güvenli ve doğrulanabilir ortak gösterim şablonuna (UIR) indirger.

2. ENTEGRE EDILEN DIL PARADIGMALARI VE SEVIYELERI

Dil Ailesi	Örnek Teknolojiler	Cerebro Ağındaki Fonksiyonu
1. Klasik Programlama	Rust, Python, Go, C++, Solidity	Sistem çekirdeği, API entegrasyonları ve MetaMask cüzdan el sıkışması.
2. Kuantum Programlama	OpenQASM, Q#, Qiskit	Lattice tabanlı Kyber-1024 koruması ve foton sapma tespiti.
3. ASI Yabancılaştırılmış Semantik	Emergent Vectorial, Agentic GNN	Yapay zeka ajanlarının kendi aralarındaki mikro-saniyeli şifreli bilgi takası.
4. Biyometrik & Nöral	EEG/EMG Akışları, Yerel Biyometrik Hash	Kullanıcının fiziksel kimliğini ve canlılık durumunu doğrulayan nöral veri akışı.

3. İKI AŞAMALI GÜVENLI WEB3 GIRIŞ VE AKORDEON KONSOL TASARIMI

Siber sızma olasılıklarını tamamen sıfırlamak adına iki kritik UI/UX ve güvenlik standardı uygulanmıştır:

- **Sızma Korumalı Cüzdan Bağlantısı:** Kullanıcı cüzdanını (MetaMask vb.) bağladığında sisteme otomatik giriş yetkisi verilmez. Cüzdan adresi üst barda sadece "Bağlı" olarak listelenir. Panel içeriği kilitli kalır. Giriş yetkisi için kullanıcının biyometrik imzası veya yerel güvenlik anahtarıyla imzalanmış ikinci aşama (Two-Factor Challenge) istenir.
- **Hiyerarşik Dikey Akordeon Menü:** Dashboard üzerindeki tüm kritik güvenlik ve analiz modülleri (Biyometrik Kimlik Doğrulama, Sertifikalar, KYC, Acil Durum Müdahalesi vb.) dikey bir akordeon yapıda alt alta butona tıklandığında iç içe açılacak şekilde yerleştirilmiştir.

4. BIRLEŞTIRILMIŞ ÜRETIM SÜRÜMÜ KAYNAK KOD YAPISI

4.1. Çoklu Dil Semantik Motoru ve Kyber-1024 Modülü (Python Çekirdeği)

```
import os
import json
from hashlib import sha256
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

class CerebroUnifiedLanguageCenter:
    def __init__(self):
        self.supported_paradigms = ["CLASSICAL", "QUANTUM", "ASI_EMERGENT", "BIOMETRIC"]
        print("[*] Cerebro Unified Language Center Başlatıldı.")

    def ingest_language_stream(self, paradigm: str, payload: dict) -> str:
        if paradigm not in self.supported_paradigms:
            raise ValueError(f"[!] Desteklenmeyen paradigma: {paradigm}")
        serialized = json.dumps(payload, sort_keys=True)
        return sha256(f"CULC_{paradigm}_{serialized}".encode()).hexdigest()

    def generate_unified_master_proof(self, class_hash, q_hash, asi_hash, bio_hash) -> str:
        master_vector = f"{class_hash}:{q_hash}:{asi_hash}:{bio_hash}"
        master_proof = sha256(f"CEREBRO_MASTER_PROOF_{master_vector}".encode()).hexdigest().upper()
        return f"ZKP-MASTER-{master_proof[:24]}"

class CerebroKyber1024AES:
    def generate_lattice_keypair(self) -> tuple:
        private_seed = os.urandom(32)
        public_seed = os.urandom(32)
        public_key = sha256(public_seed + b"CEREBRO_LATTICE_A").hexdigest()
        private_key = sha256(private_seed + b"CEREBRO_LATTICE_S").hexdigest()
        return public_key, private_key

    def encrypt_identity_data(self, data: str, shared_secret: bytes) -> tuple:
        aesgcm = AESGCM(shared_secret)
        nonce = os.urandom(12)
        encrypted_data = aesgcm.encrypt(nonce, data.encode('utf-8'), None)
        return encrypted_data.hex(), nonce.hex()
```

4.2. Otonom Bağışıklık Kalkanı ve ZKP İptal Altyapısı (JavaScript/ES6)

```
const crypto = require('crypto');

class CerebroZKPController {
    constructor() {
        this.activeProofs = new Map();
        this.networkThreatLevel = 0.05;
    }

    generateZKPProof(identityData, secretWitness) {
        const timestamp = Date.now();
        const salt = crypto.randomBytes(16).toString('hex');
        const proofHash = crypto.createHash('sha384')
            .update(identityData + secretWitness + salt + timestamp).digest('hex');
        const proofId = `zkp_${crypto.randomBytes(8).toString('hex')}`;
        const ghostId = `GHOST-ID-${crypto.createHash('sha256').update(proofHash).digest('hex').substring(0,
16).toUpperCase()}`;

        this.activeProofs.set(proofId, { proofHash, salt, timestamp, ghostId, status: "ACTIVE" });
        return { proofId, ghostId };
    }
```

```
    detectQuantumSniffing(sensorAnomalies) {
        if (sensorAnomalies > 0.45) {
            this.networkThreatLevel = 0.95;
            this.triggerSelfDestruct();
        }
    }

    triggerSelfDestruct() {
        console.warn("[!] Kuantum Sniffing Saptandı! Ephemeral Proof Purge ve Ghost Mode Aktif.");
        for (let [proofId, proof] of this.activeProofs.entries()) {
            this.activeProofs.set(proofId, {
                proofHash: crypto.randomBytes(48).toString('hex'), // Decoy / Sahte veri enjeksiyonu
                salt: null,
                ghostId: "GHOST-EXPIRED",
                status: "DECOY_ACTIVATED"
            });
        }
    }
}
```

5. KUANTUM GHOST ID ZAMAN DAMGALI PDF SERTIFIKASYON AKIŞI

Cerebro CULC aracılığıyla entegre edilen tüm dillerin ortak doğrulaması olan "ZKP-MASTER" kanıtı, üretilen her PDF sertifikasının içine görünmez dijital filigran ve şifreli bir QR kod katmanı olarak yerleştirilir. Harici ağlardaki sızma veya kopyalama girişimlerinde, dolanıklık ağındaki düğümler (Entangled Nodes) anormallik sezip sertifikayı anında sahte veri (Decoy) moduna geçirerek kendini korumaya alır.